

Data Structures Used

This project is built around a few data structures that each solve a different problem. Some help with fast lookup, some help with graph traversal, and some are just a clean way to move data from the database into the GUI.

1. Custom Unordered Map

File:

- `App/data_access/_my_unoredred_map.h`

What it is:

This is our self-built unordered map. It is used as a middle lookup layer for stadium searching instead of depending on the STL ordered map or STL unordered map for that part.

What it stores:

- a key, such as a stadium-related string
- a value, such as an ID or lookup result

How it works:

- it hashes the key
- the hash decides which bucket to use
- each bucket stores collisions with a linked list
- if the map gets too full, it rehashes into a bigger bucket array

Why we used it:

The goal was to have a fast existence check before continuing to the database side. In practice, the stadium lookup flow checks this map first. If the key is present, the program continues the lookup through the database using that mapped result instead of doing a blind search.

2. Dynamic Arrays with `std::vector`

Files:

- `App/services/trip_planner.cpp`
- `App/data_access/stadium_repository.cpp`

- `App/model/shopping_cart.cpp`
- `App/data_access/_my_unordered_map.h`

What it is used for:

- storing lists of stadiums returned from SQL
- storing trip routes
- storing selected trip targets
- storing cart items
- storing the bucket table inside the custom unordered map

Why we used it:

`vector` is a good fit when we want indexed access, simple iteration, and compact storage. A lot of the project naturally works with lists: a list of stadiums, a list of trip stops, a list of souvenirs in the cart.

3. Linked Lists inside the Custom Map Buckets

File:

- `App/data_access/_my_unordered_map.h`

What it is:

Each bucket in the custom unordered map stores nodes linked together with a `next` pointer.

Why we used it:

This handles collisions in a clean way. If two keys hash to the same bucket, we do not lose either one. We just chain them together and search that short list when needed.

Why it makes sense here:

This was the simplest way to build our own map layer while keeping insert, search, and erase logic manageable.

4. Priority Queue

File:

- `App/services/trip_planner.cpp`

What it is used for:

- Dijkstra-style shortest path work
- choosing the next best distance candidate efficiently

Why we used it:

A priority queue always lets us pull the current smallest-distance item first. That is exactly what shortest-path logic needs.

Where it helps:

- shortest trip from Dodger Stadium
- custom ordered trip segments
- custom efficient trip planning
- visit-all-by-nearest behavior

5. Queue

File:

- `App/services/trip_planner.cpp`

What it is used for:

- BFS traversal from Target Field

Why we used it:

Breadth-first search is naturally first-in, first-out. A queue matches that behavior directly and keeps the traversal easy to follow.

6. Stack

File:

- `App/services/trip_planner.cpp`

What it is used for:

- DFS traversal from Oracle Park

Why we used it:

Depth-first search needs last-in, first-out behavior. A stack makes that behavior straightforward.

7. Unordered Sets

File:

- `App/services/trip_planner.cpp`

What it is used for:

- tracking visited stadiums
- tracking remaining stadiums

- avoiding repeated work in graph traversal

Why we used it:

When we need to answer “have we already visited this stadium?” quickly, a set is much better than scanning a whole list every time.

8. SQL Tables in SQLite

Files:

- `App/data_access/database_manager.cpp`
- `App/data/BaseBall_data.db`

Main tables:

- `stadiums`
- `souvenirs`
- `stadium_distances`

Why we used them:

The database is the persistent storage layer. It keeps all stadium data, souvenir data, and graph edge data between executions. This matters because the project requirements say the changes must remain saved after the program closes.

What role it plays:

- stores long-term data
- supports sorting and filtering queries
- stores imported admin updates

9. Shopping Cart List

Files:

- `App/model/shopping_cart.h`
- `App/model/shopping_cart.cpp`

What it is:

The cart stores `CartItem` objects in a vector.

What each item holds:

- souvenir information
- quantity

Why we used it:

The cart only needs to support a simple set of operations:

- add item
- remove item
- count total quantity
- total the cost overall
- total the cost by stadium

A vector keeps this logic simple and easy to maintain.

10. Route and Result Containers

Files:

- `data_types.h`
- `App/model/trip.h`
- `App/services/trip_planner.cpp`

What they store:

- final stadium order
- transit flags
- total distance
- total cost

Why we used them:

Trip planning produces a lot of related data that needs to move together from the service layer to the trip detail page. Grouping these values into `TripResult` keeps the code cleaner than passing many separate variables around.

Summary

The project does not rely on one giant structure to do everything. Instead, each structure was picked for a job it handles well.

- The custom unordered map handles fast key-based lookup checks before stadium data is pulled from the database.
- The database handles persistent storage and sorted/filterable records.
- Vectors hold result lists and cart contents.
- The priority queue supports shortest-path logic.
- The queue supports BFS.
- The stack supports DFS.

- Sets help prevent repeated visits and duplicate work.

This combination fits the project well because the app is really doing three different kinds of work at once:

1. storing baseball data,
2. traversing a stadium graph,
3. showing clean results in a GUI.